

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Student Name: Seiitkhan Zhannur

Student ID: 230109009

Hardware / CPU: 12th Gen Intel(R) Core(™) i5 - 12450H

Date & Section: 01.10.2026

Challenge 1: The Core Speedometer & Amdahl's Law

Hardware Threads Detected: 12

Threads	Time (s)	Speedup	Efficiency (%)
---------	----------	---------	----------------

1	2.8796	1.00	x 100.0
2	1.1858	2.43	x 121.4
4	0.6577	4.38	x 109.5
8	0.5039	5.71	x 71.4
12	0.4280	6.73	x 56.1

A. $T_1 = 2.8796$ s, $T_{\text{max}} (12 \text{ threads}) = 0.4280$ s, speedup = $6.73\times$.

B. The first call compiles the function, which takes time. The warm-up does this compiling before the timer starts, so only the real work is measured.

C. No. Efficiency fell to 56% at 12 threads. My CPU's 12 threads share fewer physical cores (hyperthreading), the CPU slows its clock when all cores are busy, and starting threads adds overhead. It was above 100% at 2–4 threads because the 1-thread run was a little slow (the CPU was still speeding up).

D. Numba makes it a reduction: each thread counts in its own private copy, and the copies are added together at the end. This is the same as OpenMP `reduction(+:inside_circle)`, so threads never write to the same variable.

Challenge 2: Load Imbalance & Dynamic Scheduling

Row-Parallel Render Time: 4.208 s

Column-Parallel Render Time: 3.509 s

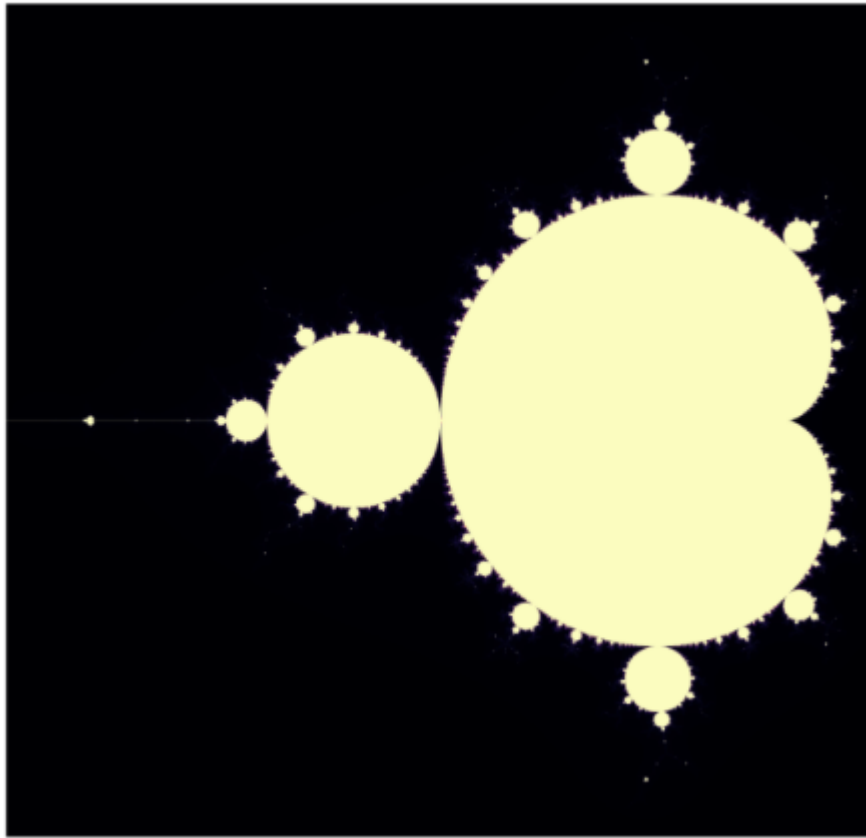
A. Columns were faster (3.509 s vs 4.208 s). Each pixel does a lot of math but only one memory write, so cache effects are small. The column split gave the threads more equal work. Also, rows ran first, while the CPU was still speeding up.

B. NumPy stores arrays row by row. Writing along a row uses each 64-byte cache line 16 times, which is fast. Writing down a column needs a new cache line for every write, which is slow, and threads also fight over shared cache lines (false sharing).

C. The top rows are outside the set and finish in 2–3 iterations, so thread 0 sits idle. The center rows run all 1000 iterations, so thread 2 is slow, and everyone waits for it. Dynamic scheduling splits the rows into small chunks, and each free thread takes the next chunk, so all threads finish at about the same time.

D.

Mandelbrot 2500x2500 (Render: 4.21s)



Challenge 3: Stencil Computation & Memory Bandwidth

Data type: float64

Heat Diffusion Complete: 0.456 s

Throughput: 1480.57 Megacells/sec

Data type: float32

Heat Diffusion Complete: 0.361 s

Throughput: 1872.18 Megacells/sec

A. Time = 0.456 s, throughput = 1480.57 Megacells/sec (float64).

B. The runtime went from 0.456 s to 0.361 s, so it got **1.26× faster** (throughput went up from 1481 to 1872 Mcells/s). This code does very little math per cell but moves a lot of data, so memory speed is the limit, not the CPU. float32 halves the data, so less data moves through RAM

and cache, and the code runs faster. It was not a full $2\times$ because some of the data already fits in the CPU cache, and starting the threads 300 times (once per step) adds a fixed cost.

C. All cores share one memory bus with a fixed speed. A few cores are already enough to use the whole bus. After that, extra cores just wait for data, so $2\times$ the cores does not give $2\times$ the speed. The memory speed is the "roof".